Python and SQLite Tutorial

Building a Student Database Step by Step

What we are building

A database is an organized collection of data. SQLite keeps a whole database in a single file, and Python can work with it through the sqlite3 package, which comes with Python, so there is nothing to install.

In this tutorial you will write one Python program, Sql.py, that creates a database file called school.db. Inside it, the program makes a table called students, fills it with eight students, and then asks the database questions about them. The students table has these columns:

Column	Data type	What it holds
id	INT	A number that identifies the student. It is the primary key, so no two students share one.
name	VARCHAR(30)	The student's first name. It cannot be empty.
age	INT	The student's age.
major	VARCHAR(20)	The subject the student studies.
credits	INT	How many credits the student has earned.
email	VARCHAR(40)	The student's email address. It can be empty.

These are the eight students the program adds. An empty email is stored as NULL, and Python shows it as None.

id	name	age	major	credits	email
1	Maya	18	Biology	15	maya@example.edu
2	Leo	21	History	45	None
3	Ana	19	Biology	30	ana@example.edu
4	Sam	22	Math	60	None
5	Priya	20	Computer Science	40	priya@example.edu
6	Omar	18	Math	12	omar@example.edu
7	Mia	23	History	75	mia@example.edu
8	Jon	19	Computer Science	28	None

Before you start

- You need Python 3 and any editor that can run Python files, such as IDLE or VS Code.
- Type each piece of code exactly as shown. Spelling, quotes and brackets matter.
- Lines inside a for loop start with four spaces. Each step says when a line is indented.
- Lines that start with # are comments. Python ignores them, so they are optional.

1. Create the Python file

In Module 5 folder, create a new file, and save it with this name:

```
Sql.py
```

The database file will be created in the same folder, Module 5.

2. Import the package

Type this first line. It loads the sqlite3 package so your program can use it.

```
import sqlite3
```

3. Connect to the database

Leave a blank line, then type these two lines. connect opens school.db, or creates it if it does not exist yet. The cursor is the object that sends SQL commands to the database.

```
conn = sqlite3.connect("school.db")   # opens school.db, or creates it
cur = conn.cursor()                   # the cursor runs SQL commands
```

4. Close the connection

A program should close the database when it is finished. Add these two lines now. Keep them at the very bottom of the file, and type every new piece of code above them from now on.

```
conn.close()
print("\nDone.")
```

Save and run the program. You will see a blank line, then:

Output:

```
Done.
```

Look in your folder: a new file called school.db is there. That file is your database.

5. Remove any old table

Where: below the cursor line, above conn.close().

DROP TABLE deletes a table. IF EXISTS means no error happens if the table is not there yet. This line lets you run the program again and again, because each run starts with a fresh table.

```
# ---------- Create the table ----------
cur.execute("DROP TABLE IF EXISTS students")
```

6. Create the table

Where: right below the DROP TABLE line.

CREATE TABLE makes the students table. The SQL is long, so it goes inside triple quotes, which let a Python string cover several lines. NOT NULL means a column must have a value, and PRIMARY KEY marks the column that identifies each row.

```
cur.execute("""
    CREATE TABLE students
    (   id       INT         NOT NULL,
        name     VARCHAR(30) NOT NULL,
        age      INT,
        major    VARCHAR(20),
        credits  INT,
        email    VARCHAR(40),
        PRIMARY KEY (id) )
""")
print("Table created.")
```

Save and run the program. Above Done., you will now also see:

Output:

```
Table created.
```

7. Add one student

Where: below the CREATE TABLE block.

INSERT INTO adds a row. The values go in the same order as the columns. Text goes inside single quotes; numbers do not. Like CREATE TABLE, the SQL sits inside triple quotes so it can be split across lines.

```
# ---------- Add rows ----------
cur.execute("""
    INSERT INTO students
    VALUES (1, 'Maya', 18, 'Biology', 15, 'maya@example.edu')
""")
```

8. Add many students at once

Where: below the INSERT block you just typed.

For many rows, put the data in a Python list of tuples. executemany runs the INSERT once for each tuple, and each ? is filled in from the tuple, in order. None becomes NULL, an empty value. The SQL is first stored in a variable called query, which keeps the line short.

```
students = [
    (2, "Leo",   21, "History",          45, None),
    (3, "Ana",   19, "Biology",          30, "ana@example.edu"),
    (4, "Sam",   22, "Math",             60, None),
    (5, "Priya", 20, "Computer Science", 40, "priya@example.edu"),
    (6, "Omar",  18, "Math",             12, "omar@example.edu"),
    (7, "Mia",   23, "History",          75, "mia@example.edu"),
    (8, "Jon",   19, "Computer Science", 28, None),
]
query = "INSERT INTO students VALUES (?, ?, ?, ?, ?, ?)"
cur.executemany(query, students)
```

Each line inside the square brackets starts with four spaces.

9. Save the changes

Where: below the executemany line.

Changes to the data are not saved until you call commit. Without it, the new rows are lost when the program ends.

```
conn.commit()   # save the changes to the file
print("Students added.")
```

Save and run the program. Above Done., you will now also see:

Output:

```
Students added.
```

10. Read every row

Where: below the commit lines.

SELECT * FROM students asks for every column of every row. fetchall returns the rows as a list, with one tuple per row, and the for loop prints them one at a time. The \n at the start of the message prints a blank line first, to separate the results.

```
# ---------- Read rows ----------
print("\nEvery student:")
cur.execute("SELECT * FROM students")
rows = cur.fetchall()
for row in rows:
    print(row)
```

The line print(row) starts with four spaces, because it is inside the for loop.

Save and run the program. Above Done., you will now also see:

Output:

```
Every student:
(1, 'Maya', 18, 'Biology', 15, 'maya@example.edu')
(2, 'Leo', 21, 'History', 45, None)
(3, 'Ana', 19, 'Biology', 30, 'ana@example.edu')
(4, 'Sam', 22, 'Math', 60, None)
(5, 'Priya', 20, 'Computer Science', 40, 'priya@example.edu')
(6, 'Omar', 18, 'Math', 12, 'omar@example.edu')
(7, 'Mia', 23, 'History', 75, 'mia@example.edu')
(8, 'Jon', 19, 'Computer Science', 28, None)
```

11. Choose columns and filter rows

Where: below the previous loop.

Name the columns you want after SELECT. WHERE keeps only the rows that meet a condition, here an age of 20 or more.

```
print("\nStudents aged 20 or older:")
cur.execute("SELECT name, age FROM students WHERE age >= 20")
for row in cur.fetchall():
    print(row)
```

Again, print(row) is indented by four spaces.

Save and run the program. Above Done., you will now also see:

Output:

```
Students aged 20 or older:
('Leo', 21)
('Sam', 22)
('Priya', 20)
('Mia', 23)
```

12. Use a Python variable in a query

Where: below the previous loop.

To put a value from a variable into SQL, write a ? where the value goes, and pass the value in a tuple. A tuple with one item needs a comma after it: (major,).

```
major = "Biology"
print("\nStudents in", major)
cur.execute("SELECT name FROM students WHERE major = ?", (major,))
for row in cur.fetchall():
    print(row)
```

Try changing "Biology" to "Math" and run the program again.

Save and run the program. Above Done., you will now also see:

Output:

```
Students in Biology
('Maya',)
('Ana',)
```

13. Combine conditions: AND, OR, NOT

Where: below the previous loop.

From here on, the code uses a shorter pattern. As in step 8, the SQL is stored in the variable query, and the for loop goes straight over cur.execute(query), so fetchall is not needed. AND needs both conditions to be true, OR needs at least one, and NOT reverses a condition.

```
# ---------- Filter rows ----------
print("\nAND: Biology students under 19")
query = "SELECT name FROM students WHERE major = 'Biology' AND age < 19"
for row in cur.execute(query):
    print(row)

print("\nOR: Math students, or anyone with over 50 credits")
query = "SELECT name FROM students WHERE major = 'Math' OR credits > 50"
for row in cur.execute(query):
    print(row)

print("\nNOT: students not in History")
query = "SELECT name FROM students WHERE NOT major = 'History'"
for row in cur.execute(query):
    print(row)
```

Each print(row) line is indented by four spaces.

Save and run the program. Above Done., you will now also see:

Output:

```
AND: Biology students under 19
('Maya',)

OR: Math students, or anyone with over 50 credits
('Sam',)
('Omar',)
('Mia',)

NOT: students not in History
('Maya',)
('Ana',)
('Sam',)
('Priya',)
('Omar',)
('Jon',)
```

14. Match lists, patterns and empty values: IN, LIKE, IS NULL

Where: below the NOT loop.

IN checks whether a value is in a list. LIKE matches a pattern, where % stands for any number of characters, so 'M%' means starts with M. IS NULL finds empty values.

```
print("\nIN: students in Biology or History")
query = "SELECT name FROM students WHERE major IN ('Biology', 'History')"
for row in cur.execute(query):
    print(row)

print("\nLIKE: names starting with M")
query = "SELECT name FROM students WHERE name LIKE 'M%'"
for row in cur.execute(query):
    print(row)

print("\nIS NULL: students with no email")
query = "SELECT name FROM students WHERE email IS NULL"
for row in cur.execute(query):
    print(row)
```

Save and run the program. Above Done., you will now also see:

Output:

```
IN: students in Biology or History
('Maya',)
('Leo',)
('Ana',)
('Mia',)

LIKE: names starting with M
('Maya',)
('Mia',)

IS NULL: students with no email
('Leo',)
('Sam',)
('Jon',)
```

15. Remove duplicates and sort: DISTINCT, ORDER BY

Where: below the IS NULL loop.

DISTINCT shows each value only once. ORDER BY sorts the result from smallest to largest, or A to Z, and DESC sorts from largest to smallest.

```
# ---------- Sort and summarize ----------
print("\nDISTINCT: each major once")
query = "SELECT DISTINCT major FROM students"
for row in cur.execute(query):
    print(row)

print("\nORDER BY: names from A to Z")
query = "SELECT name FROM students ORDER BY name"
for row in cur.execute(query):
    print(row)

print("\nORDER BY ... DESC: most credits first")
query = "SELECT name, credits FROM students ORDER BY credits DESC"
for row in cur.execute(query):
    print(row)
```

Save and run the program. Above Done., you will now also see:

Output:

```
DISTINCT: each major once
('Biology',)
('History',)
('Math',)
('Computer Science',)

ORDER BY: names from A to Z
('Ana',)
('Jon',)
('Leo',)
('Maya',)
('Mia',)
('Omar',)
('Priya',)
('Sam',)

ORDER BY ... DESC: most credits first
('Mia', 75)
('Sam', 60)
('Leo', 45)
('Priya', 40)
('Ana', 30)
('Jon', 28)
('Maya', 15)
('Omar', 12)
```

16. Summarize: COUNT, SUM, AVG, MIN, MAX

Where: below the last loop, still above conn.close().

These functions turn many rows into one result. The SQL is in triple quotes again, and it returns a single row, so fetchone is enough, and the five values are unpacked into five variables. round(average, 1) keeps one decimal place.

```
cur.execute("""
    SELECT COUNT(*), SUM(credits), AVG(age), MIN(age), MAX(age)
    FROM students
""")
count, total, average, youngest, oldest = cur.fetchone()
print("\nNumber of students:", count)
print("Total credits:", total)
print("Average age:", round(average, 1))
print("Youngest:", youngest, " Oldest:", oldest)
```

Save and run the program. Above Done., you will now also see:

Output:

```
Number of students: 8
Total credits: 305
Average age: 20.0
Youngest: 18  Oldest: 23
```

17. The complete program

Your Sql.py should now look like this. If something does not work, compare your file with it line by line.

```
import sqlite3

conn = sqlite3.connect("school.db")   # opens school.db, or creates it
cur = conn.cursor()                   # the cursor runs SQL commands

# ---------- Create the table ----------
cur.execute("DROP TABLE IF EXISTS students")

cur.execute("""
    CREATE TABLE students
    (   id       INT         NOT NULL,
        name     VARCHAR(30) NOT NULL,
        age      INT,
        major    VARCHAR(20),
        credits  INT,
        email    VARCHAR(40),
        PRIMARY KEY (id) )
""")
print("Table created.")

# ---------- Add rows ----------
cur.execute("""
    INSERT INTO students
    VALUES (1, 'Maya', 18, 'Biology', 15, 'maya@example.edu')
""")

students = [
    (2, "Leo",   21, "History",          45, None),
    (3, "Ana",   19, "Biology",          30, "ana@example.edu"),
    (4, "Sam",   22, "Math",             60, None),
    (5, "Priya", 20, "Computer Science", 40, "priya@example.edu"),
    (6, "Omar",  18, "Math",             12, "omar@example.edu"),
    (7, "Mia",   23, "History",          75, "mia@example.edu"),
    (8, "Jon",   19, "Computer Science", 28, None),
]
query = "INSERT INTO students VALUES (?, ?, ?, ?, ?, ?)"
cur.executemany(query, students)

conn.commit()   # save the changes to the file
print("Students added.")

# ---------- Read rows ----------
print("\nEvery student:")
cur.execute("SELECT * FROM students")
rows = cur.fetchall()
for row in rows:
    print(row)
```

```
print("\nStudents aged 20 or older:")
cur.execute("SELECT name, age FROM students WHERE age >= 20")
for row in cur.fetchall():
    print(row)

major = "Biology"
print("\nStudents in", major)
cur.execute("SELECT name FROM students WHERE major = ?", (major,))
for row in cur.fetchall():
    print(row)

# ---------- Filter rows ----------
print("\nAND: Biology students under 19")
query = "SELECT name FROM students WHERE major = 'Biology' AND age < 19"
for row in cur.execute(query):
    print(row)

print("\nOR: Math students, or anyone with over 50 credits")
query = "SELECT name FROM students WHERE major = 'Math' OR credits > 50"
for row in cur.execute(query):
    print(row)

print("\nNOT: students not in History")
query = "SELECT name FROM students WHERE NOT major = 'History'"
for row in cur.execute(query):
    print(row)

print("\nIN: students in Biology or History")
query = "SELECT name FROM students WHERE major IN ('Biology', 'History')"
for row in cur.execute(query):
    print(row)

print("\nLIKE: names starting with M")
query = "SELECT name FROM students WHERE name LIKE 'M%'"
for row in cur.execute(query):
    print(row)

print("\nIS NULL: students with no email")
query = "SELECT name FROM students WHERE email IS NULL"
for row in cur.execute(query):
    print(row)

# ---------- Sort and summarize ----------
print("\nDISTINCT: each major once")
query = "SELECT DISTINCT major FROM students"
for row in cur.execute(query):
    print(row)
```

```
print("\nORDER BY: names from A to Z")
query = "SELECT name FROM students ORDER BY name"
for row in cur.execute(query):
    print(row)

print("\nORDER BY ... DESC: most credits first")
query = "SELECT name, credits FROM students ORDER BY credits DESC"
for row in cur.execute(query):
    print(row)

cur.execute("""
    SELECT COUNT(*), SUM(credits), AVG(age), MIN(age), MAX(age)
    FROM students
""")
count, total, average, youngest, oldest = cur.fetchone()
print("\nNumber of students:", count)
print("Total credits:", total)
print("Average age:", round(average, 1))
print("Youngest:", youngest, " Oldest:", oldest)

conn.close()
print("\nDone.")
```

18. Try it yourself

Add each answer above conn.close(), run the program, and check your result. Try each one on your own before looking at the answer.

Exercise 1

Show the name and major of every student with more than 30 credits, sorted by name.

```
print("\nMore than 30 credits:")
query = "SELECT name, major FROM students WHERE credits > 30 ORDER BY name"
for row in cur.execute(query):
    print(row)
```

Exercise 2

Store a major in a variable, then count how many students have that major. Use a ? placeholder.

```
wanted = "Computer Science"
cur.execute("SELECT COUNT(*) FROM students WHERE major = ?", (wanted,))
print("Students in", wanted, ":", cur.fetchone()[0])
```

Output of both answers:

```
More than 30 credits:
('Leo', 'History')
('Mia', 'History')
('Priya', 'Computer Science')
('Sam', 'Math')
Students in Computer Science : 2
```